

## B. TECH II YEAR I SEM --- BACKEND WEB TECHNOLOGIES

### UNIT-1

#### SHORT QUESTION AND ANSWERS:

##### 1.Explain briefly about Routing?

**Routing** in Express is the process of defining how an application responds to client requests for different URLs (routes) and HTTP methods such as **GET, POST, PUT, and DELETE**. Each route is associated with a handler function that processes the request and sends a response.

Syntax:

```
</> JavaScript
app.METHOD(PATH, HANDLER);
```

Example:

```
</> JavaScript
app.get('/', (req, res) => {
  res.send('Welcome to Express');
});
```

Here, when a client sends a **GET** request to the **/** route, the server responds with **"Welcome to Express"**.

##### 2. What is mean by Server?

A **server** is a computer or software application that receives requests from clients, processes them, and sends back appropriate responses over a network. It provides services such as web pages, data, or files to users.

##### Example:

An **Express.js server** listens for client requests and responds to them.

```
</> JavaScript

const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello, World!');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

### 3.What is mean by Backend development?

**Backend development** is the process of building the **server-side** part of a web application. It is responsible for handling client requests, implementing business logic, interacting with databases, and sending responses to the client.

#### **Example:**

When a user logs in, the backend verifies the username and password, retrieves data from the database, and sends the result back to the frontend.

**Technologies Used:** Node.js, Express.js, MongoDB, MySQL, PostgreSQL, etc.

### 4.What is the importance of Express.js?

**Express.js** is a lightweight and flexible **Node.js web framework** used to build web applications and APIs quickly. It simplifies server-side development by providing features like routing, middleware support, and easy handling of HTTP requests and responses.

#### **Importance of Express.js:**

- Simplifies web application development.
- Provides powerful routing for handling URLs.
- Supports middleware for request processing.
- Makes it easy to build RESTful APIs.
- Improves code organization and maintainability.

### 5.Explain briefly about Node.js.

**Node.js** is an **open-source, cross-platform JavaScript runtime environment** that allows JavaScript to run outside the browser. It is mainly used to develop **server-side applications** and **REST APIs**. Node.js uses the **V8 JavaScript Engine** and follows an **event-driven, non-blocking I/O model**, making it fast and efficient.

#### **Key Features:**

- Runs JavaScript outside the browser.
- Uses the V8 JavaScript Engine.
- Supports event-driven, non-blocking I/O.

- Suitable for building scalable web applications and APIs.

## 6.What is the purpose of handling HTTP requests in web applications?

**Handling HTTP requests** allows a web application to receive requests from clients (such as browsers), process them on the server, and send appropriate responses. It enables communication between the client and the server for performing operations like retrieving, creating, updating, and deleting data.

### Examples of HTTP methods:

- **GET** – Retrieve data.
- **POST** – Create new data.
- **PUT** – Update existing data.
- **DELETE** – Remove data.

#### Example:

JavaScript

```
const express = require('express');
const app = express();

app.get('/students', (req, res) => {
  res.send('List of Students');
});

app.listen(3000);
```

## LONG QUESTION AND ANSWERS:

### 1.Explain Backend Development. Discuss its role in web application architecture?

**Backend Development** is the process of developing the **server-side** part of a web application. It is responsible for handling client requests, implementing business logic, interacting with databases, authenticating users, and sending appropriate responses to the frontend. Backend development ensures that the application functions correctly behind the scenes.

### Role of Backend Development in Web Application Architecture

The backend plays a vital role in the architecture of a web application by performing the following functions:

#### 1. Handles Client Requests

- Receives HTTP requests (GET, POST, PUT, DELETE) from the frontend.
- Processes the request and sends an appropriate response.

#### 2. Implements Business Logic

- Executes the application's rules and operations.
- Example: Calculating discounts, validating login credentials, or checking product availability.

#### 3. Database Management

- Stores, retrieves, updates, and deletes data from databases such as MongoDB, MySQL, or PostgreSQL.
- Ensures data consistency and integrity.

#### 4. User Authentication and Authorization

- Verifies user identity using login credentials.
- Controls access to resources based on user roles and permissions.

#### 5. API Development

- Creates REST APIs that allow the frontend and other applications to communicate with the server.

## 6. Security

- Protects sensitive data through encryption, authentication, input validation, and secure communication.

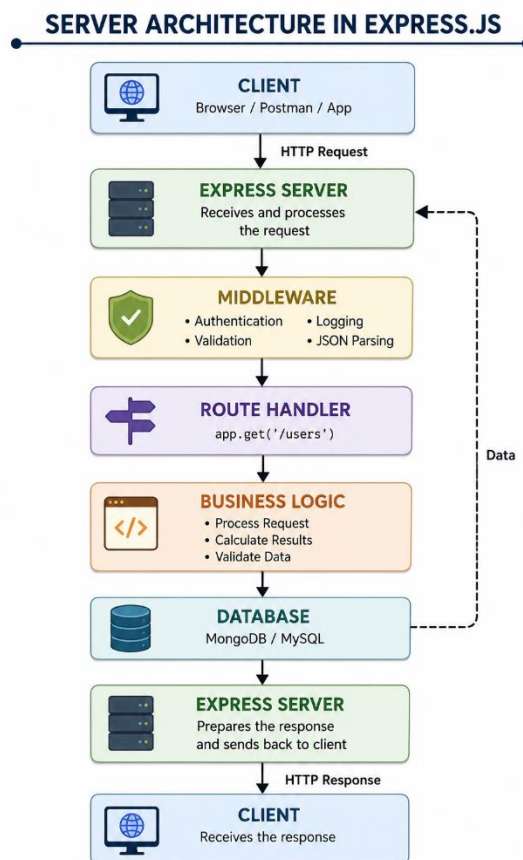
## 7. Performance and Scalability

- Handles multiple client requests efficiently.
- Optimizes application performance for large numbers of users.

### 2.Explain Server Architecture in detail and discuss its major features?

**Server Architecture in Express.js** is a structure that explains how an Express application processes client requests. It follows a simple **Client → Server → Middleware → Route → Business Logic → Database → Response** architecture.

When a client sends an HTTP request, the Express server processes it through different layers and finally returns an appropriate response.



## Components of Server Architecture

### 1. Client

- The client sends an HTTP request to the Express server.
- Examples:
  - Postman
  - Frontend Application

### 2. Express Server

- Receives client requests.
- Processes requests and sends responses.
- Built using the Express framework.

### 3. Middleware Layer

- Middleware functions execute before the request reaches the route.
- Used for:
  - Logging
  - Authentication
  - Validation

### 4. Routing Layer

- Routes determine which function handles the incoming request.

```
</> JavaScript
app.get('/users', (req, res) => {
  res.send("Users Data");
});
```

### 5. Business Logic Layer

- Contains the application's main functionality.
- Performs tasks such as:

- Processing data
- Calculations
- Input validation

## 6. Database Layer

- Stores and retrieves application data.
- Common databases:
  - MongoDB
  - MySQL

## 7. Response

- After processing the request, the server sends an HTTP response back to the client.
- Responses may contain JSON, HTML, or error messages.

```
JavaScript

const express = require("express");

const app = express();

// Middleware
app.use((req, res, next) => {
  console.log("Request received");
  next();
});

// Route
app.get("/hello", (req, res) => {

  // Business Logic
  const message = "Hello, Express.js!";

  // Response
  res.send(message);
});

// Start Server
app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

### 3. Explain CommonJS and ESM modules in detail

#### Introduction

JavaScript modules are used to organize code into separate files so that it becomes easier to develop, maintain, and reuse. Node.js supports two module systems:

1. **CommonJS (CJS)**
  2. **ES Modules (ESM)**
- 

#### 1. CommonJS (CJS)

**CommonJS** is the default module system used in Node.js. It uses the `require()` function to import modules and `module.exports` (or `exports`) to export them.

##### Syntax

math.js

```
</> JavaScript   
  
function add(a, b) {  
  return a + b;  
}  
  
module.exports = add;
```

app.js

```
</> JavaScript   
  
const add = require('./math');  
  
console.log(add(10, 20));
```

#### Features of CommonJS

- Default module system in Node.js.
- Uses `require()` to import modules.
- Uses `module.exports` to export modules.
- Loads modules synchronously.
- Suitable for server-side applications.

## 2. ES Modules (ESM)

**ES Modules (ECMAScript Modules)** are the standard JavaScript module system supported by modern browsers and Node.js. They use the `import` and `export` keywords.

### Syntax

math.js

```
</> JavaScript
export function add(a, b) {
  return a + b;
}
```



app.js

```
</> JavaScript
import { add } from './math.js';

console.log(add(10, 20));
```



### Features of ES Modules

- Standard JavaScript module system.
- Uses `import` and `export`.
- Supports static analysis and tree shaking.
- Supported in modern browsers and Node.js.
- Better suited for modern web applications.

| Feature          | CommonJS (.js)                                | ES6 Modules (.mjs)                         |
|------------------|-----------------------------------------------|--------------------------------------------|
| Default Export   | <code>module.exports = fn</code>              | <code>export default fn</code>             |
| Named Export     | <code>exports.fn = fn</code>                  | <code>export const fn = ...</code>         |
| Import (default) | <code>const fn = require('./file')</code>     | <code>import fn from './file.mjs'</code>   |
| Import (named)   | <code>const { fn } = require('./file')</code> | <code>import {fn} from './file.mjs'</code> |
| File extension   | <code>.js</code>                              | <code>.mjs</code>                          |

#### 4. Briefly explain about Error Handling in Express.js.

**Error Handling** in Express.js is the process of detecting, catching, and managing errors that occur while processing client requests. It ensures that the server does not crash and returns a meaningful error response to the client.

One of the most common ways to handle errors in Express.js is by using the **try...catch** statement.

---

##### How try...catch Works

- The **try** block contains code that may generate an error.
- If an error occurs, control is transferred to the **catch** block.
- The **catch** block handles the error and sends an appropriate response to the client.

Syntax:

```
</> JavaScript   
  
try {  
    // Code that may throw an error  
} catch (error) {  
    // Handle the error  
}
```

## Example

</> JavaScript

```
const express = require("express");
const app = express();

app.get("/students", (req, res) => {
  try {
    // Business Logic
    const students = [
      { id: 1, name: "Rahul" },
      { id: 2, name: "Priya" }
    ];

    res.json(students);

  } catch (error) {
    res.status(500).json({
      message: "Internal Server Error"
    });
  }
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

## Working

1. The client sends a request to **/students**.
2. The server executes the code inside the **try** block.
3. If no error occurs, the student data is returned.
4. If an error occurs, the **catch** block catches the error.
5. The server sends an error response with **HTTP Status Code 500**.

## Advantages of Error Handling

- Prevents the server from crashing.
- Improves application reliability.
- Returns meaningful error messages to the client.
- Makes debugging easier.
- Enhances the user experience.

## 5.Explain NPM Package Management in detail. Discuss its features and commands

**NPM (Node Package Manager)** is the default package manager for **Node.js**. It is used to install, update, remove, and manage packages (libraries or modules) required for a Node.js application. NPM also helps manage project dependencies using the package.json file.

### Features of NPM

#### 1. Package Management

- Installs and manages third-party packages.

#### 2. Dependency Management

- Automatically installs all required dependencies for a project.

#### 3. Version Management

- Supports different versions of packages and updates them when needed.

#### 4. package.json Support

- Stores project information and dependency details.

#### 5. Large Package Registry

- Provides access to millions of open-source packages through the NPM registry.

#### 6. Script Execution

- Allows developers to define and run scripts such as start and test.

| Command                 | Purpose                                                                 |
|-------------------------|-------------------------------------------------------------------------|
| npm init                | <u>Creates a new Node.js project and generates a package.json file.</u> |
| npm install             | <u>Installs all dependencies listed in package.json.</u>                |
| npm install <package>   | <u>Installs a specific package.</u>                                     |
| npm install express     | <u>Installs the Express.js package.</u>                                 |
| npm uninstall <package> | <u>Removes a package from the project.</u>                              |
| npm update              | <u>Updates installed packages to newer versions.</u>                    |

|           |                                                                     |
|-----------|---------------------------------------------------------------------|
| npm list  | <u>Displays the installed packages.</u>                             |
| npm start | <u>Runs the application using the start script in package.json.</u> |

## Example

### Step 1: Create a Project

```
</> Bash  
npm init -y
```



### Step 2: Install Express.js

```
</> Bash  
npm install express
```



### Step 3: Install SQLite Packages

```
</> Bash  
npm install sqlite sqlite3
```



### Step 4: Start the Application

```
</> Bash  
npm start
```



## Advantages of NPM

- Simplifies package installation and management.
- Saves project dependencies in package.json.
- Automatically installs required packages.
- Makes code reusable through third-party libraries.
- Provides access to a large collection of open-source packages.

## 6.Explain briefly about Request and Response Objects in Express.js

In **Express.js**, the **Request (req)** and **Response (res)** objects are used to handle communication between the client and the server.

- The **Request (req)** object contains information sent by the client, such as URL, parameters, query strings, request body, and headers.
- The **Response (res)** object is used by the server to send data or status codes back to the client.

### Request (req) Object

The **req** object represents the HTTP request received from the client.

### Common Properties

- **req.params** – Retrieves route parameters.
- **req.query** – Retrieves query parameters.
- **req.body** – Retrieves data sent in the request body.
- **req.headers** – Retrieves request headers.
- **req.method** – Returns the HTTP method (GET, POST, PUT, DELETE).
- **req.url** – Returns the requested URL.

### Example

```
<> JavaScript   
app.get("/students/:id", (req, res) => {  
  console.log(req.params.id);  
  res.send("Student Details");  
});
```

If the request is:

```
GET /students/101 
```

Output:

```
101 
```

## Response (res) Object

The **res** object is used to send a response from the server to the client.

### Common Methods

- **res.send()** – Sends text or HTML.
- **res.json()** – Sends JSON data.
- **res.status()** – Sets the HTTP status code.

### Example

</> JavaScript 

```
app.get("/users", (req, res) => {  
  res.json([  
    { id: 1, name: "Ali" },  
    { id: 2, name: "Sara" }  
  ]);  
});
```

Output:

</> JSON 

```
[  
  { "id": 1, "name": "Ali" },  
  { "id": 2, "name": "Sara" }  
]
```

## UNIT-2

### SHORT QUESTION AND ANSWERS:

#### 1. Define SQLite. Mention any two features of SQLite.

SQLite is a **lightweight, serverless, open-source relational database management system (RDBMS)** that stores the entire database in a **single file**. It is widely used for developing web, mobile, and desktop applications.

#### Features (Any Two):

1. **Serverless:** SQLite does not require a separate database server.
2. **Lightweight:** It is small in size, fast, and easy to install and use.

#### 2. What is a Database Schema?

A **Database Schema** is the **logical structure or blueprint of a database**. It defines how the data is organized, including the **tables, columns, data types, relationships, and constraints**.

#### Example Schema

| Column Name | Data Type |
|-------------|-----------|
| student_id  | INTEGER   |
| name        | TEXT      |
| age         | INTEGER   |
| course      | TEXT      |

#### SQL Query to Create the Schema

```
</> SQL
CREATE TABLE students (
  student_id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  course TEXT
);
```

#### Explanation

- **CREATE TABLE** → Creates a new table.

- student\_id INTEGER PRIMARY KEY → Creates the primary key column.
- name TEXT → Stores the student's name.
- age INTEGER → Stores the student's age.
- course TEXT → Stores the course name.

This query defines the **database schema** for the students table.

### 3.Expand CRUD and list its operations.

**CRUD** stands for:

- **C – Create:** Used to insert new data into a database.
- **R – Read:** Used to retrieve or view data from a database.
- **U – Update:** Used to modify existing data in a database.
- **D – Delete:** Used to remove data from a database.

### SQL Operations

| CRUD Operation                 | SQL Command | SQLite Method | Purpose                                        |
|--------------------------------|-------------|---------------|------------------------------------------------|
| <b>Create</b>                  | INSERT      | db.run()      | Inserts a new record into the table.           |
| <b>Read (Single Record)</b>    | SELECT      | db.get()      | Retrieves one record from the table.           |
| <b>Read (Multiple Records)</b> | SELECT      | db.all()      | Retrieves all matching records from the table. |
| <b>Update</b>                  | UPDATE      | db.run()      | Updates existing records in the table.         |
| <b>Delete</b>                  | DELETE      | db.run()      | Deletes records from the table.                |

## 4. What is a RESTful API?

A **RESTful API (Representational State Transfer Application Programming Interface)** is an API that follows the **REST architectural principles** to enable communication between a client and a server using **HTTP methods** such as **GET, POST, PUT, and DELETE**.

A RESTful API allows clients to perform **CRUD (Create, Read, Update, Delete)** operations on resources.

### Example

| HTTP Method | Endpoint    | Purpose                  |
|-------------|-------------|--------------------------|
| GET         | /students   | Retrieve all students    |
| POST        | /students   | Add a new student        |
| PUT         | /students/1 | Update student with ID 1 |
| DELETE      | /students/1 | Delete student with ID 1 |

### Express.js Example

JavaScript

```
const express = require("express");
const app = express();

app.get("/students", (req, res) => {
  res.send("List of Students");
});

app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

## 5. Differentiate between Query Parameters and Route Parameters.

| Route Parameters                      | Query Parameters                      |
|---------------------------------------|---------------------------------------|
| Used to identify a specific resource. | Used to filter, sort, or search data. |
| Included as part of the URL path.     | Added after the ? in the URL.         |
| Accessed using req.params.            | Accessed using req.query.             |
| Example: /students/101                | Example: /students?course=CSE         |

### Express.js Example

JavaScript

```
// Route Parameter
app.get("/students/:id", (req, res) => {
  res.send(req.params.id);
});

// Query Parameter
app.get("/students", (req, res) => {
  res.send(req.query.course);
});
```

### Example URLs

- Route Parameter:

http://localhost:3000/students/101

Output:

101

- Query Parameter:

http://localhost:3000/students?course=CSE

Output:

CSE

## 6. What is the purpose of a Primary Key in a database?

A **Primary Key** is a column (or a combination of columns) that **uniquely identifies each record (row)** in a database table. It ensures that no two rows have the same value and does not allow **NULL** values.

### Purpose of a Primary Key

- **Uniquely identifies** each record in a table.
- **Prevents duplicate** records.
- **Does not allow NULL values.**
- Helps establish relationships between tables.

### SQL Query

SQL



```
CREATE TABLE students (  
  student_id INTEGER PRIMARY KEY,  
  name TEXT,  
  age INTEGER  
);
```

## LONG QUESTION AND ANSWERS:

### 1.Explain SQLite integration in Node.js with suitable code examples.

SQLite integration in Node.js is the process of connecting a Node.js application with an SQLite database to perform database operations such as **Create, Read, Update, and Delete (CRUD)**. The sqlite and sqlite3 packages are commonly used to establish the connection and execute SQL queries.

#### Steps to Integrate SQLite with Node.js

##### Step 1: Install Required Packages

```
</> Bash  
npm install sqlite sqlite3
```

##### Step 2: Import Required Modules

```
</> JavaScript  
const express = require("express");  
const path = require("path");  
const { open } = require("sqlite");  
const sqlite3 = require("sqlite3");
```

- express – Creates the web server.
- path – Specifies the database file path.
- open () – Opens the SQLite database.
- sqlite3 – SQLite database driver.

##### Step 3: Connect to the SQLite Database

JavaScript

```
const app = express();

const dbPath = path.join(__dirname, "students.db");

let db = null;

const initializeDBAndServer = async () => {
  try {
    db = await open({
      filename: dbPath,
      driver: sqlite3.Database,
    });

    app.listen(3000, () => {
      console.log("Server Running at http://localhost:3000/");
    });

  } catch (error) {
    console.log(`DB Error: ${error.message}`);
    process.exit(1);
  }
};

initializeDBAndServer();
```

### Explanation:

- open() establishes the connection to the SQLite database.
- If the connection is successful, the Express server starts.
- If an error occurs, the catch block displays the error message and terminates the application.

### Advantages of SQLite Integration

- Easy to integrate with Node.js.
- No separate database server is required (serverless).
- Lightweight and fast.
- Suitable for small and medium-sized applications.
- Supports CRUD operations efficiently.

## 2. Describe CRUD operations in Node.js using SQLite with examples.

**CRUD** stands for **Create, Read, Update, and Delete**. These are the four basic operations performed on data stored in a database. In **Node.js**, SQLite provides the methods **db.run()**, **db.get()**, and **db.all()** to perform CRUD operations.

---

### CRUD Operations

#### 1. Create (INSERT)

The **Create** operation is used to insert a new record into a database table. It uses the **INSERT** SQL statement and the **db.run()** method.

#### Example:

JavaScript

```
const query = `
INSERT INTO students (id, name, age)
VALUES (1, 'Rahul', 20);
`;

await db.run(query);
```

#### 2. Read (SELECT)

The **Read** operation is used to retrieve data from a database.

- **db.get()** retrieves a single record.
- **db.all()** retrieves multiple records.

#### Example (Single Record):

JavaScript

```
const query = `
SELECT * FROM students
WHERE id = 1;
`;

const student = await db.get(query);
console.log(student);
```

#### Example (Multiple Records):

JavaScript

```
const query = `
SELECT * FROM students;
`;

const students = await db.all(query);
console.log(students);
```

### 3. Update (UPDATE)

The **Update** operation is used to modify existing records in a database table. It uses the **UPDATE** SQL statement and the **db.run()** method.

**Example:**

JavaScript

```
const query = `
UPDATE students
SET age = 21
WHERE id = 1;
`;

await db.run(query);
```

### 4. Delete (DELETE)

The **Delete** operation is used to remove records from a database table. It uses the **DELETE** SQL statement and the **db.run()** method.

**Example:**

JavaScript

```
const query = `
DELETE FROM students
WHERE id = 1;
`;

await db.run(query);
```

## SQLite Methods Used

| Operation  | SQL Command | SQLite Method |
|------------|-------------|---------------|
| Create     | INSERT      | db.run()      |
| Read (One) | SELECT      | db.get()      |
| Read (All) | SELECT      | db.all()      |
| Update     | UPDATE      | db.run()      |
| Delete     | DELETE      | db.run()      |

## Advantages of CRUD Operations

- Enables efficient data management.
- Supports adding, retrieving, updating, and deleting records.
- Makes applications interactive and dynamic.
- Simplifies database operations using SQLite methods.

## 3. Explain Query Writing and Query Optimization techniques in Node.js.

**Answer:**

### Definition

**Query Writing** is the process of writing **SQL statements** to perform database operations such as **Create, Read, Update, and Delete (CRUD)**. In **Node.js**, SQL queries are executed using SQLite methods like **db.run()**, **db.get()**, and **db.all()**.

**Query Optimization** is the process of improving SQL queries so they execute faster and use fewer system resources.

---

## 1. Query Writing

Query writing involves creating SQL statements to interact with the database.

### Example 1: Retrieve All Students

retrieve all students

```
JavaScript

const query = `
SELECT * FROM students;
`;

const students = await db.all(query);
```

## Example 2: Retrieve a Student by ID

```
JavaScript

const studentId = req.params.id;

const query = `
SELECT * FROM students
WHERE id = ${studentId};
`;

const student = await db.get(query);
```

## Example 3: Insert a Record

```
JavaScript

const query = `
INSERT INTO students(name, age)
VALUES ('Rahul', 20);
`;

await db.run(query);
```

## 2. Query Optimization Techniques

### 1. Retrieve Only Required Columns

Instead of retrieving all columns, select only the required columns.

### ✖ Not Optimized

SQL

```
SELECT * FROM students;
```



### ✔ Optimized

SQL

```
SELECT name, age FROM students;
```



## 2. Use the WHERE Clause

Retrieve only the required records.

SQL

```
SELECT * FROM students  
WHERE id = 1;
```



## 3. Use LIMIT

Retrieve only a limited number of rows.

SQL

```
SELECT * FROM students  
LIMIT 5;
```



## 4. Use ORDER BY When Needed

Sort the results in ascending or descending order.

SQL

```
SELECT * FROM students  
ORDER BY name ASC;
```



## 5. Use Primary Keys for Searching

Searching using a **Primary Key** is faster because it uniquely identifies each record.

 SQL

```
SELECT * FROM students  
WHERE id = 1;
```



### Advantages of Query Optimization

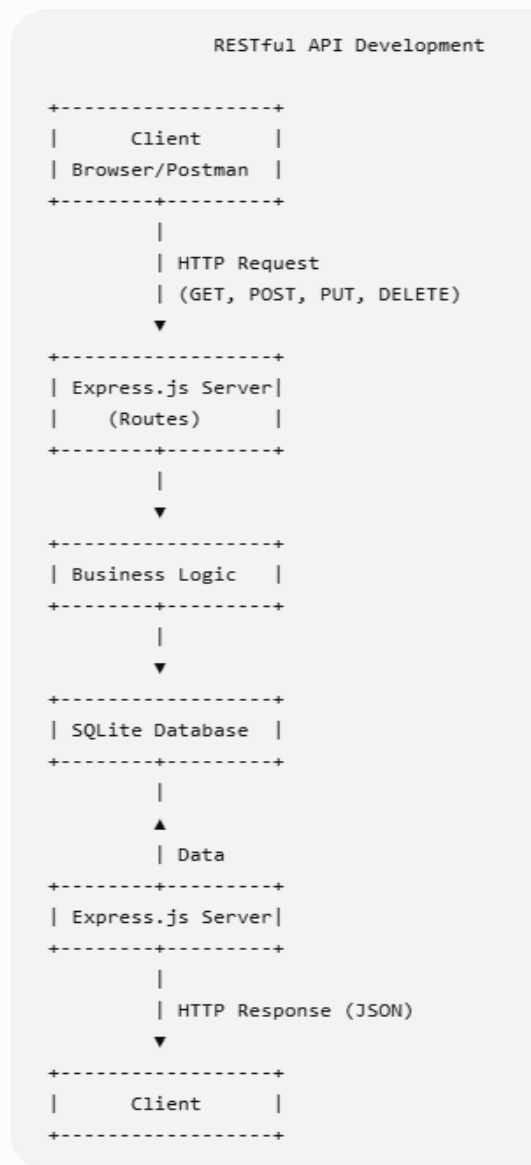
- Improves query execution speed.
- Reduces database load.
- Retrieves only the required data.
- Improves application performance.
- Reduces memory and CPU usage.

### 4. Explain RESTful API Development in Node.js using Express.js with an example.

A **RESTful API** (Representational State Transfer Application Programming Interface) is an API that follows the **REST architectural principles** to enable communication between the client and the server using **HTTP methods** such as GET, POST, PUT, and DELETE.

In **Node.js**, the **Express.js** framework is widely used to develop RESTful APIs because it provides simple routing, middleware support, and request-response handling.

## RESTful API Architecture



### HTTP Methods Used in RESTful APIs

| HTTP Method | Operation | Description                     |
|-------------|-----------|---------------------------------|
| GET         | Read      | Retrieves data from the server. |
| POST        | Create    | Adds new data to the database.  |
| PUT         | Update    | Modifies existing data.         |
| DELETE      | Delete    | Removes data from the database. |

### Example: Student RESTful API

```
</> JavaScript

const express = require("express");

const app = express();

app.use(express.json());

// GET - Read all students
app.get("/students", (req, res) => {
  res.json([
    { id: 1, name: "Rahul" },
    { id: 2, name: "Priya" }
  ]);
});

// POST - Add a new student
app.post("/students", (req, res) => {
  const student = req.body;
  res.send("Student Added Successfully");
});

// PUT - Update student
app.put("/students/:id", (req, res) => {
  const { id } = req.params;
  res.send(`Student ${id} Updated Successfully`);
});

// DELETE - Delete student
app.delete("/students/:id", (req, res) => {
  const { id } = req.params;
  res.send(`Student ${id} Deleted Successfully`);
});

app.listen(3000, () => {
  console.log("Server Running on Port 3000");
});
```

### 5. Discuss Query Handling in Express.js using query parameters?

**Query handling** in Express.js is the process of retrieving and processing **query parameters** sent by the client in the URL. Query parameters are **key-value pairs** that appear after the **?** symbol in the URL. They are commonly used for searching, filtering, sorting, and pagination. In Express.js, query parameters are accessed using the **req.query** object.

## 1.1 Filtering Books

- Get a specific number of books
- Get books based on search query text
- Get books in the sorted order

### 1.1.1 Get a specific number of books

To get specific number of books in certain range we use **limit** and **offset**.

**Offset** is used to specify the **position** from where rows are to be selected.

**Limit** is used to specify the **number of rows**.

and many more...

Query parameters starts with ? (question mark) followed by key value pairs separated by & (ampersand)

### 1.1.2 Get books based on search query text

We provide query text to **search\_q** key

```
1 search_q = potter
```

### 1.1.3 Get books in the sorted order

We provide sorted order to **order** key

Ascending: **ASC**

DESCENDING: **DESC**

```
1 order = ASC
2 order = DESC
```

## Filtering GET Books API

```
1- app.get("/books/", async (request, response) => {
2-   const {
3-     offset = 2,
4-     limit = 5,
5-     order = "ASC",
6-     order_by = "book_id",
7-     search_q = "",
8-   } = request.query;
9-   const getBooksQuery = `
10-    SELECT
11-      *
12-    FROM
13-      book
14-    WHERE
15-      title LIKE '%${search_q}%'
16-    ORDER BY ${order_by} ${order}
17-    LIMIT ${limit} OFFSET ${offset}`;
18-   const booksArray = await db.all(getBooksQuery);
19-   response.send(booksArray);
20- });
```

JAVASCRIPT

Collapse ^

## 6. Explain the REST Principles in detail with suitable examples.

**REST** (Representational State Transfer) principles provide a standard way of designing web APIs. By following REST principles, applications become scalable, reliable, maintainable, and easy to integrate with different clients such as web browsers, mobile applications, and desktop applications.

### Importance of REST Principles

#### 1. Scalability

- REST APIs can handle a large number of client requests efficiently.
- Since REST is **stateless**, the server does not store client information between requests, making it easier to add more servers when the number of users increases.

#### Example:

An e-commerce website like Amazon can handle thousands of users searching for products simultaneously because each request is processed independently.

---

#### 2. Reliability

- Every request contains all the information needed to process it.
- If one request fails, it does not affect other requests.

#### Example:

If a request to retrieve student details fails, other users can still access the application without interruption.

---

#### 3. Simplicity

- REST uses standard HTTP methods such as GET, POST, PUT, and DELETE.
  - Developers can easily understand and build REST APIs.
- 

#### 4. Platform Independence

- REST APIs can be accessed from different platforms such as:

- Web applications
  - Mobile applications
  - Desktop applications
- 

## 5. Better Performance

- REST supports lightweight data formats like JSON, which reduces network traffic and improves response time.

## 2.2 REST API Principles

### 1. Providing a Unique ID to Each Resource

In REST, every resource must have a **unique URL (Uniform Resource Locator)** so that it can be identified and accessed individually.

#### Example

`/students`



Returns all students.

`/students/101`



Returns the student whose ID is **101**.

`/books/10`



Returns the book whose ID is **10**.

### Express.js Example

</> JavaScript

```
app.get("/students/:id", (req, res) => {  
  const { id } = req.params;  
  res.send(`Student ID: ${id}`);  
});
```

#### Request

GET /students/101

#### Response

Student ID: 101

## 2. Using Standard HTTP Methods

REST APIs use standard HTTP methods to perform CRUD operations.

| HTTP Method | Operation            | Example Endpoint |
|-------------|----------------------|------------------|
| GET         | Retrieve data        | /students        |
| POST        | Create new data      | /students        |
| PUT         | Update existing data | /students/101    |
| DELETE      | Delete data          | /students/101    |

## 3. Accept and respond with JSON

REST APIs commonly use **JSON (JavaScript Object Notation)** as the format for exchanging data between the client and server.

### Why JSON?

- Lightweight.
- Easy for humans to read.
- Easy for machines to parse.
- Supported by almost all programming languages.

## UNIT-III

- **SHORT QUESTION AND ANSWERS:**

### **1. Define JWT Token Authentication?**

**JWT (JSON Web Token) Authentication** is a **authentication mechanism** used to securely verify the identity of a user. After a successful login, the server generates a **JSON Web Token (JWT)** and sends it to the client. The client stores the token and includes it in the **Authorization** header of every subsequent request. The server validates the token before allowing access to protected resources.

### **How JWT Authentication Works**

#### **1. User Login**

- The user sends login credentials (username and password) to the server.

#### **2. Credential Verification**

- The server verifies the username and password using the database.

#### **3. JWT Token Generation**

- If the credentials are valid, the server generates a JWT token.

#### **4. Token Storage**

- The client stores the token (e.g., in Local Storage or an HTTP-only cookie).

#### **5. Sending the Token**

- The client includes the token in the **Authorization** header with every protected request.

**Authorization: Bearer <JWT\_Token>**

#### **6. Token Verification**

- The server verifies the JWT token.
- If the token is valid, access is granted; otherwise, access is denied with an **Unauthorized (401)** response.

## 2.What is the purpose of middleware?

Middleware is a special kind of function in Express JS which accepts the request from

- the user (or)
- the previous middleware

After processing the request the middleware function

- sends the response to another middleware (or)
- calls the API Handler (or)
- sends response to the user

```
1 app.method(Path, middleware1, handler);
```

JAVASCRIPT

Example

```
1 const jsonMiddleware = express.json();
2 app.use(jsonMiddleware);
```

JAVASCRIPT

It is a **built-in middleware function** it recognizes the incoming request object as a JSON object, parses it, and then calls handler in **every API call**

### 1.1 Multiple Middleware functions

We can pass multiple middleware functions

```
1 app.method(Path, middleware1, middleware2, handler);
```

JAVASCRIPT

## 3. List any two features of a User Management System.

A **User Management System** is an application used to manage user accounts, authentication, and access permissions.

### Any Two Features

1. **User Registration and Login** – Allows users to create an account and securely log in using their credentials.

2. **Role-Based Access Control (RBAC)** – Assigns roles (such as **Admin**, **User**, or **Seller**) and controls access to resources based on those roles.

**Other features include:**

- Password encryption (hashing)
- Profile management
- JWT authentication
- Password reset
- User authorization

## LONG QUESTION AND ANSWERS:

1. **Explain JWT Token Authentication and demonstrate its implementation in Node.js using Express.js.**

**JWT (JSON Web Token) Authentication** is a **stateless authentication mechanism** used to securely verify the identity of a user. After a successful login, the server generates a **JWT token** and sends it to the client. The client stores the token and sends it in the **Authorization** header with every subsequent request. The server verifies the token before allowing access to protected resources.

---

### How JWT Authentication Works

#### Step 1: User Login

- The user sends their **username** and **password** to the server.

#### Step 2: Verify Credentials

- The server verifies the user's credentials from the database.

#### Step 3: Generate JWT Token

- If the credentials are valid, the server generates a JWT token.

#### Step 4: Send Token to Client

- The server sends the generated token to the client.

#### Step 5: Store the Token

- The client stores the JWT token (Local Storage or HTTP-only Cookie).

#### Step 6: Send Token with Every Request

- The client includes the token in the **Authorization** header.

Authorization: Bearer <JWT\_Token>

#### Step 7: Verify the Token

- The server verifies the token.
- If valid, access is granted; otherwise, a **401 Unauthorized** response is returned.

## Implementation in Node.js using Express.js

### Step 1: Install JWT Package

 Bash



```
npm install jsonwebtoken
```

### Step 2: Import Required Modules

 JavaScript



```
const express = require("express");  
const jwt = require("jsonwebtoken");
```



```
const app = express();  
  
app.use(express.json());
```

### Step 3: Generate JWT Token (Login Route)

 JavaScript



```
app.post("/login", (req, res) => {  
  
  const user = {  
    id: 1,  
    username: "Rahul"  
  };  
  
  const payload = {  
    userId: user.id,  
    username: user.username  
  };  
  
  const jwtToken = jwt.sign(payload, "MY_SECRET_KEY");  
  
  res.send({ jwtToken });  
});
```

### Step 4: Verify JWT Token (Middleware)

 JavaScript 

```
const authenticateToken = (req, res, next) => {  
  
  const authHeader = req.headers["authorization"];  
  
  const jwtToken = authHeader && authHeader.split(" ")[1];  
  
  if (!jwtToken) {  
    return res.status(401).send("Invalid JWT Token");  
  }  
  
  jwt.verify(jwtToken, "MY_SECRET_KEY", (error, payload) => {  
  
    if (error) {  
      return res.status(401).send("Invalid JWT Token");  
    }  
  
    req.user = payload;  
    next();  
  });  
};
```



## Step 5: Access Protected Route

 JavaScript 

```
app.get("/profile", authenticateToken, (req, res) => {  
  res.send("Welcome " + req.user.username);  
});
```

## Step 6: Start the Server

 JavaScript 

```
app.listen(3000, () => {  
  console.log("Server Running on Port 3000");  
});
```

## 2. Explain the User Management process in Node.js with suitable examples.

User needs to be registered and then log in to access the books

- Register User API
- Login User API

### 2.1 Register User API

Here we check whether the user is a new user or an existing user. Returns "User Already exists" for existing user else for a new user we store encrypted password in the DB

```
1 app.post("/users/", async (request, response) => {
2   const { username, name, password, gender, location } = request.body;
3   const hashedPassword = await bcrypt.hash(request.body.password, 10);
4   const selectUserQuery = `SELECT * FROM user WHERE username = '${username}'`;
5   const dbUser = await db.get(selectUserQuery);
6   if (dbUser === undefined) {
7     const createUserQuery = `
8       INSERT INTO
9         user (username, name, password, gender, location)
10      VALUES
11      (
12        '${username}',
13        '${name}',
14        '${hashedPassword}',
15        '${gender}',
16        '${location}'
17      );
18     const dbResponse = await db.run(createUserQuery);
19     const newUserId = dbResponse.lastID;
20     response.send(`Created new user with ${newUserId}`);
21   } else {
22     response.status = 400;
23     response.send("User already exists");
24   }
25 });
```

### 2.2 Login User API

Here we check whether the user exists in DB or not. Returns "Invalid User" if the user doesn't exist else we compare the given password with the DB user password

```

1 app.post("/login", async (request, response) => {
2   const { username, password } = request.body;
3   const selectUserQuery = `SELECT * FROM user WHERE username = '${username}'`;
4   const dbUser = await db.get(selectUserQuery);
5   if (dbUser === undefined) {
6     response.status(400);
7     response.send("Invalid User");
8   } else {
9     const isPasswordMatched = await bcrypt.compare(password, dbUser.password);
10    if (isPasswordMatched === true) {
11      response.send("Login Success!");
12    } else {
13      response.status(400);
14      response.send("Invalid Password");
15    }
16  }
17 });

```

### 3. Describe Middleware Functions in Express.js and explain their implementation with code examples.

**Middleware** is a function in **Express.js** that executes **between the client's request and the server's response**. It has access to the **request object (req)**, **response object (res)**, and the **next middleware function (next)**.

Middleware can:

- Process incoming requests.
- Modify the request or response objects.
- Execute additional logic such as logging or authentication.
- End the request-response cycle.
- Pass control to the next middleware using `next()`.

#### Syntax

```

</> JavaScript
function middleware(req, res, next) {
  // Perform some task
  next(); // Pass control to the next middleware
}

```

#### Types of Middleware in Express.js

1. Application-level Middleware
2. Router-level Middleware
3. Built-in Middleware
4. Authentication Middleware

## 1. Application-Level Middleware

```
</> JavaScript   
  
const express = require("express");  
const app = express();  
  
function logger(req, res, next) {  
  console.log("Request Received");  
  next();  
}  
  
app.use(logger);  
  
app.get("/", (req, res) => {  
  res.send("Home Page");  
});  
  
app.listen(3000);
```

## 2. Built-in Middleware

Express provides built-in middleware such as `express.json()` to parse JSON request bodies.

```
</> JavaScript   
  
const express = require("express");  
const app = express();  
  
app.use(express.json());  
  
app.post("/user", (req, res) => {  
  res.send(req.body);  
});  
  
app.listen(3000);
```

### 3. Authentication Middleware

Authentication middleware verifies whether a user is logged in before allowing access to protected routes.

JavaScript

```
const express = require("express");
const app = express();

function authenticate(req, res, next) {
  const isLoggedIn = true;

  if (isLoggedIn) {
    next();
  } else {
    res.send("Access Denied");
  }
}

app.get("/profile", authenticate, (req, res) => {
  res.send("Welcome to Profile");
});

app.listen(3000);
```

### 4. Router-Level Middleware

Router-level middleware works only for a specific router.

JavaScript

```
const express = require("express");
const app = express();

function checkUser(req, res, next) {
  console.log("Checking User");
  next();
}

app.get("/dashboard", checkUser, (req, res) => {
  res.send("Dashboard");
});

app.listen(3000);
```